

M3 · SELECTIVE REPEAT

Sohini Pandit

Roll 2405913 · GitHub `sohinix` · CSE-14 · Reviewer: Sohan Mandal (Sprint A), Shaili Seth (Sprint B)

You build the smartest of the three protocols: when one packet is lost, only that packet is sent again. The receiver must hold packets that arrive early and release them in order once the gap fills, and every packet in flight has its own timer. It is the hardest module in the project. That is why your Sprint A builds its hardest piece — the receive buffer — as a plain data structure before any networking, why you own the sequence arithmetic both window protocols share, and why there is a fallback decision on Tuesday 22 so the project can never stall on one module.

Fri 18 Sep

Receive buffer and `SeqSpace` merged

Tue 22 Sep, 10 pm

Fallback decision with Sohan

Thu 24 Sep

Selective Repeat merged — Core complete depends on it

YOU DEPEND ON	DEPEND ON YOU
Sinjan — <code>TimerWheel</code> on Fri 18	Sinjan — <code>SeqSpace</code> for Go-Back-N's wraparound
Shaili — <code>Packet</code> on Thu 17; <code>ArqProtocol</code> and conventions on Sat 19	Shaili — Selective Repeat for the integrity suite, Thu 24
Sohan — the emulator on Thu 17	Everyone — Core complete, Thu 24
	Sohan — Experiment 2 runs from Fri 25

Contents

1 · Get set up	9 · Experiment 2
2 · Your part of the system	10 · Your report section
3 · Concepts you must master	11 · How we work together
4 · Design decisions you must defend	12 · Using AI without losing marks
5 · Your day-by-day plan	13 · Viva preparation
6 · Team conventions	14 · Help, references, glossary
7 · How to test your work	15 · Sprint checklist
8 · When something breaks	

1 · Get set up — by Tuesday 15 September, 10 pm

Gate 1 is the first check of the sprint: all four of us have cloned the repository, verified our git identity, and pushed a first commit. Everything after it depends on this.

The repository is not a shared folder. Sohan's copy lives on Sohan's laptop. You make your own complete copy on *your* laptop by cloning it from GitHub, then you work, commit and push from there. GitHub is only the meeting point. Every command in this handbook runs on your own machine. (If you already have a working clone, start at step 4.)

Step 1 — Install the tools

- **Git.** On Windows this also installs **Git Bash** — run every command in this handbook there, not in PowerShell or Command Prompt.
- **JDK 21** (Adoptium Temurin). Check that `java -version` and `javac -version` both report 21.
- **WSL2 with Ubuntu** on Windows, because the experiments run on Linux. Inside Ubuntu install the JDK and Python too:

```
wsl --install -d Ubuntu
wsl -d Ubuntu
sudo apt update && sudo apt install -y openjdk-21-jdk make python3-pip
java -version
```

Step 2 — Clone onto your laptop

```
git clone https://github.com/sohan1611/rdt-udp.git
cd rdt-udp
```

You now have your own `rdt-udp` folder. Everything from here on happens inside it.

Step 3 — Tell git who you are, inside that folder

```
git config user.name "Sohini Pandit"
git config user.email "the-email-on-your-github-account"
```

This email decides whether you count as a contributor. It must be one of the addresses listed at github.com/settings/emails for your account `sohinix`. If you would rather not use a personal address, that page shows a private one ending in `@users.noreply.github.com` that links just as well. Commits with an unrecognised email show a grey avatar and are credited to nobody — and sir treats commit history as evidence of individual contribution.

Step 4 — Confirm the build

```
./build.sh
```

You should see three test suites and **33 tests, 0 failed**. If anything fails, paste the output in the group chat the same evening.

Step 5 — Push your first commit

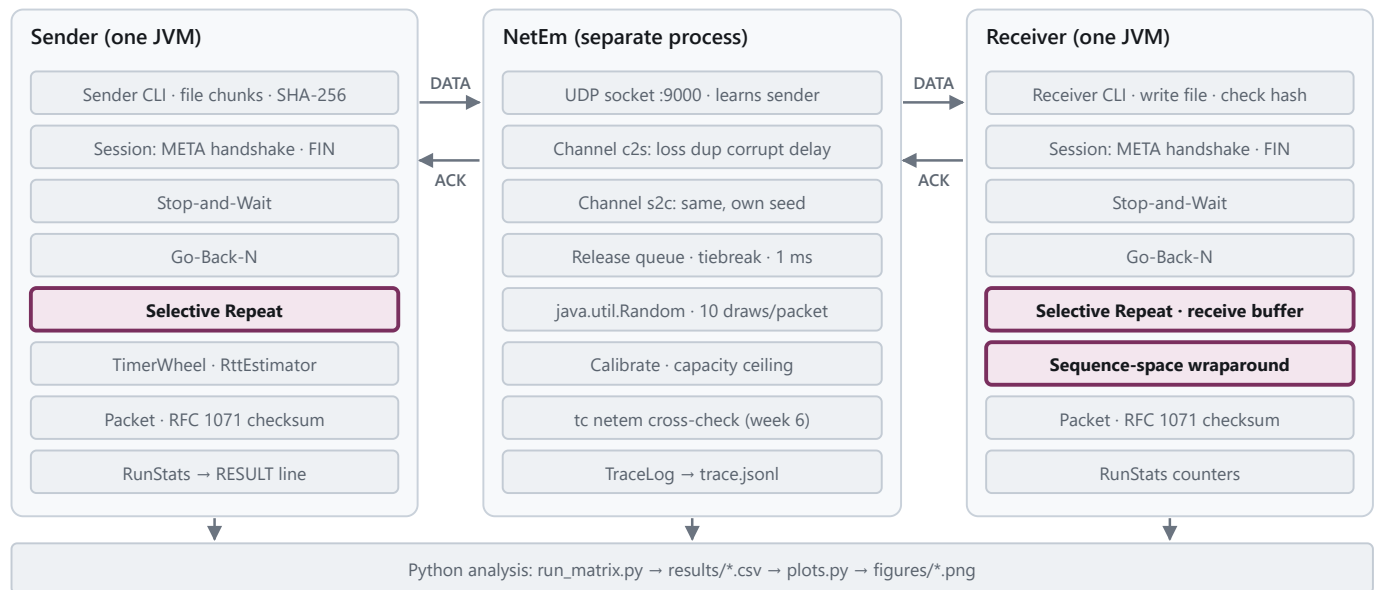
Open `AI-USE.md` and replace `__[name]_` beside your role heading with your name. Then:

```
git checkout -b setup/sohini
git add AI-USE.md
git commit -m "Add Sohini to AI-USE.md"
git push -u origin setup/sohini
git log -1 --format='%an <%ae>'
```

The last command must print your own name and your GitHub email. Open the pull request on GitHub and ask Sohan Mandal to approve it. Your first push asks you to sign in to GitHub; on Windows a browser window opens, which is normal.

Done when: your PR is merged, `git log` shows your own name and email, and you can see yourself on the repository's Contributors page.

2 · Your part of the system



Bold, coloured boxes are yours. Grey boxes belong to teammates — either you depend on them, or they depend on you. Packets always travel through NetEm: the sender and receiver never talk directly.

Files you own

FILE	STATUS	WHAT IT DOES
docs/sr-window-bound.md	New, Wed 16	Your explanation of $W \leq 2^{k-1}$, with the failure case. It opens your report section.
src/main/java/rdt/SeqSpace.java	New, Wed 16	Modular sequence arithmetic shared by both window protocols (convention 14)
src/main/java/rdt/ReceiveBuffer.java	New, Wed 16 – Fri 18	Holds early packets and releases a contiguous run in order
src/main/java/rdt/SelectiveRepeat.java	New, Sun 20 – Thu 24	The protocol: per-packet timers, selective ACKs, buffered delivery
src/test/java/rdt/SeqSpaceTest.java , ReceiveBufferTest.java , SelectiveRepeatTest.java	New, with each class	See section 7

Ask before changing Packet (Shaili), TimerWheel or GoBackN (Sinjan), or anything under emulator/ (Sohan).

3 · Concepts you must master

3.1 · The Selective Repeat sender

The window runs from `send_base` for N sequence numbers. Each slot in it is in one of three states: *sent and waiting for its ACK*, *acknowledged*, or *not yet sent*.

EVENT	SENDER ACTION
Room in the window and data left	Send the next packet and start its own timer
ACK s , inside the window	Mark s acknowledged and cancel its timer. If s is <code>send_base</code> , slide <code>send_base</code> forward past every consecutive acknowledged slot — which may let new packets go out.
ACK s , outside the window	Ignore it
Timer for s fires	If s is still unacknowledged, resend only s and restart only its timer

3.2 · The Selective Repeat receiver

The receive window runs from `rcv_base` for N sequence numbers.

AN INTACT PACKET s ARRIVES...	RECEIVER ACTION
inside the window	ACK s . Store it if not already stored. If s is <code>rcv_base</code> , deliver it and every consecutive stored packet after it, advancing <code>rcv_base</code> past them.
in the previous window, <code>rcv_base - N</code> to <code>rcv_base - 1</code>	ACK s again , but do not store or deliver it
anywhere else	Ignore it

Why the previous window must be re-ACKed. Suppose s arrived and was delivered, but its ACK was lost. The sender still believes s is missing, so its `send_base` cannot move past it, and it keeps resending s . If the receiver ignores those copies, the sender never gets its ACK and the transfer deadlocks. Re-ACKing is what unblocks it.

3.3 · A worked timeline

A four-packet file, window $N = 4$. Packet 2 is lost.

```
sender  sends 1 2 3 4, starting four timers
receiver gets 1 -> it is rcv_base: deliver 1, ACK 1, rcv_base = 2
receiver gets 3 -> early: store 3, ACK 3
receiver gets 4 -> early: store 4, ACK 4
sender  ACK 1 -> send_base = 2.  ACK 3, ACK 4 -> marked, their timers cancelled
sender  timer for 2 fires -> resend 2, and only 2
receiver gets 2 -> deliver 2, then 3 and 4 from the buffer; rcv_base = 5; ACK 2
sender  ACK 2 -> send_base slides past 3 and 4 to 5. Done.
```

Retransmissions: Selective Repeat 1 (packet 2) Go-Back-N 3 (packets 2, 3, 4)

3.4 · The window bound — the question you will be asked

With k sequence bits there are 2^k numbers, and they wrap around to 0. The receiver can never tell an *old* packet 0 from a *new* packet 0 — both carry the number 0. So the two windows must never be able to overlap:

sender window + receiver window $\leq 2^k$

Go-Back-N: receiver window is 1 -> $W + 1 \leq 2^k$ -> $W \leq 2^k - 1$

Selective Repeat: receiver window is W -> $W + W \leq 2^k$ -> $W \leq 2^{(k-1)}$

The failure, with $k = 2$ — numbers 0, 1, 2, 3:

Illegal: $W = 3$
 sender sends 0 1 2 -> all arrive and are delivered
 receiver window moves on to 3 0 1
 every ACK is lost -> sender times out and resends its OLD packet 0
 0 is inside the receiver's window -> stored as the NEW packet 0
 -> later delivered as file data. Wrong bytes, and nothing reports an error.

Legal: $W = 2$
 sender sends 0 1 -> delivered -> receiver window moves on to 2 3
 every ACK is lost -> sender resends old 0
 0 is in the previous window (0 1) -> re-ACKed, not delivered ✓

This is why the sequence-space size must be at least 8 bits for our largest experiment window of 128. We use 32 bits, so only your wraparound tests ever get close to the limit.

3.5 · Modular arithmetic in Java — and the % trap

Every window test reduces to one quantity: **how far is s ahead of the base, around the circle?**

$\text{offset}(s, \text{base}) = \text{Math.floorMod}(s - \text{base}, M) \quad M = 2^k$
 in the window $\text{offset}(s, \text{base}) < N$
 in the previous one $\text{Math.floorMod}(\text{base} - s, M)$ is between 1 and N

Never use % for this. In Java, $(1 - 3) \% 4$ is -2 , because % keeps the sign of the left side. $\text{Math.floorMod}(1 - 3, 4)$ is 2 , which is the answer you need. This single substitution is the most common wraparound bug there is.

Check yourself with $M = 8$, $\text{base} = 6$, $N = 4$, so the window is {6, 7, 0, 1}:

$s = 1$	$\text{floorMod}(1 - 6, 8) = \text{floorMod}(-5, 8) = 3$	$3 < 4$	inside the window	✓
$s = 2$	$\text{floorMod}(2 - 6, 8) = 4$	4 is not < 4	not inside	✓
$s = 4$	$\text{floorMod}(6 - 4, 8) = 2$	2 is in $1..4$	previous window {2,3,4,5}	✓

With 32-bit sequence numbers held in a `long`, $M = 2^{32} = 4,294,967,296$ fits comfortably. Put these three operations in `SeqSpace` (convention 14) so the rule is written once, tested once, and used by Sinjan's Go-Back-N too.

3.6 · Designing the receive buffer

OPTION	FOR	AGAINST
Ring buffer of N slots	$O(1)$ store and lookup; memory fixed at $N \times 1400$ bytes; indexed by <i>offset</i> , which is already the wraparound-safe number	A little more bookkeeping for the head index
TreeMap by sequence number	Very little code	Sorted by raw number, so after a wrap packet 0 sorts before 4,294,967,295 — the ordering breaks exactly where it matters

Recommendation: the ring buffer, indexed by offset from `rcv_base`. Because the index is computed with `SeqSpace`, wraparound is handled in one place. The buffer needs to: accept a packet and say whether it was *new*, a *duplicate*, *previous-window* or *outside*; hand back everything now deliverable, in order; and report the current base. Build and test it with no sockets at all.

3.7 · Per-packet timers

Each unacknowledged packet has a timer in Sinjan's `TimerWheel` — at most N at once. An ACK cancels one timer; a timeout resends one packet. **Always check the packet is still unacknowledged when its timer fires.** If the heap cancels lazily, a cancelled timer can still reach the top, and without that check you would resend a packet that was already acknowledged — which shows up as retransmissions on a perfect channel.

3.8 · ACKs and the sackCount field

Core: each ACK acknowledges one packet — the `ack` field holds its number and `sackCount` is 0 (convention 3). **Week 7 stretch — SACK blocks:** `sackCount` says how many ranges follow in the payload, each a pair of 32-bit start and end numbers. One ACK can then report everything received, which helps most when ACKs are being lost.

3.9 · The fallback, if Tuesday 22 needs it

Keep the receiver exactly as it is. On the sender, replace per-packet timers with **one timer on the oldest unacknowledged packet**; when it fires, resend only that packet. It is still selective and still correct. What it loses: when several packets in one window are lost, they are recovered one timeout at a time rather than in parallel. Document it as a deliberate simplification, and restore per-packet timers in week 7.

4 · Design decisions you must defend

DECISION	WHY
<code>W ≤ 2^{k-1}</code> , and the protocol refuses anything larger	Beyond it, old and new packets become indistinguishable. Draw 3.4.
Ring buffer indexed by offset	O(1), bounded memory, wraparound handled once. See 3.6.
A shared <code>SeqSpace</code> helper using <code>Math.floorMod</code>	The <code>%</code> trap; one tested implementation for both window protocols.
Re-ACK the previous window	Otherwise a lost ACK deadlocks the sender. See 3.2.
Check "still unacknowledged" when a timer fires	Lazy cancellation can surface stale timers. See 3.7.
Individual ACKs in Core, SACK blocks as stretch	Correctness first; SACK is an efficiency gain, not a requirement.

5 · Your day-by-day plan

Sprint A · 14–20 September

Mon 14 – Tue 15

Setup and theory

1. Complete section 1. First commit merged by Tuesday 10 pm.
2. Read Kurose & Ross section 3.4.4, including the figure where the receiver cannot tell an old packet from a new one.
3. Derive the bound in 3.4 yourself on paper, then draw the $k = 2$ failure without looking.

Done when: first commit merged, and you can draw the failure case from memory.

Be able to explain: question 1 in section 13 — the one you are most likely to be asked.

Wed 16

Window-bound note and SeqSpace

1. Commit `docs/sr-window-bound.md`: the rule, the $k = 2$ failure, the legal case.
2. `git checkout -b m3/seq-space`. Write `SeqSpace` with the three operations in 3.5, and a test for every line of the worked example there, plus the `%` trap. Open the PR the same day — Sinjan will build on it.

Done when: the note is committed and the `SeqSpace` PR is open with its tests passing.

Wed 16 – Fri 18

Receive buffer, standalone

1. `git checkout -b m3/receive-buffer`. Build the ring buffer from 3.6 with no networking.
2. Feed it 1, 3, 4, 2 and assert it releases 1, then nothing, then nothing, then 2, 3, 4 together.
3. Add duplicates of a stored packet, duplicates of a delivered packet, previous-window packets and outside packets.
4. Thursday: review Sohan's emulator PR with three *why* questions.
5. Friday: wraparound cases with a 3-bit sequence space, crossing 6, 7, 0, 1. Merge by Friday night. **8 pm: conventions call** — you own convention 14 and are consulted on convention 3.

Done when: `SeqSpace` and the buffer are merged by Friday night, passing every case in section 7.

Be able to explain: questions 5 and 6 in section 13.

Sat 19 – Sun 20

Design and start SelectiveRepeat

1. Saturday: on paper, design the sender's three slot states from 3.1 and exactly what happens on an ACK and on a timeout. Read `docs/CONVENTIONS.md` and the merged `ArqProtocol` interface once Shaili commits them.
2. Sunday: `git checkout -b m3/selective-repeat`. Build the receiver side first — it is mostly your buffer, which already works.

Sprint B · 21–27 September

Mon 21 – Tue 22

Sender, first transfers, fallback decision

1. Monday: the sender with per-packet timers from 3.1. First transfer at 0% loss through the emulator.
2. Tuesday: run at 10% loss. Use `--verbose` once to confirm that **only lost packets are resent**, never a whole window.

Tuesday 22, 10 pm — the fallback decision, with Sohan. The test: does Selective Repeat transfer a file correctly at 10% loss on at least three seeds? If yes, carry on. If no, switch the sender to the single timer in 3.9 tonight. That is still correct, still ahead of Go-Back-N, and far easier to finish by Thursday. Choosing it on Tuesday is good planning. Discovering on Thursday that you should have is what breaks the sprint.

Done when: at 10% loss the logs show only lost packets resent — or the fallback has been chosen.

Wed 23 – Thu 24

Finish · Core complete depends on you

1. Wednesday: an end-to-end run with `--seqbits 3` and window 4 that wraps several times and still matches. Add the window validation, so a window above 2^{k-1} is refused with a clear error.
2. Wednesday: the illegal-window demonstration, at buffer level — see the tip in section 7.
3. Thursday: merge by the afternoon, so Shaili can add you to the integrity suite that evening.

Done when: Selective Repeat passes the shared integrity suite — **Gate 5, Core complete**, Thursday 10 pm.

Be able to explain: questions 2, 3, 4 and 8 in section 13.

Fri 25 – Sun 27

Experiment 2

1. With Sohan, start the Go-Back-N and Selective Repeat cells from section 9.
2. Sunday: generate Figure 2 with `plots.py`.

Done when: Figure 2 is generated by script from committed CSVs with at least 3 seeds per cell.

After the sprint

28 Sep – 26 Oct

Buffer, then weeks 6–8

1. **28 Sep – 5 Oct:** Experiment 2 at the full 10 seeds; finish anything that slipped.
2. **6–12 Oct:** teach Selective Repeat at the cross-teaching session — Sohan in particular must be able to explain it. Draft your report section.
3. **13–19 Oct:** stretch goal: SACK blocks, *only* if Core is solid. If the fallback was taken, restore per-packet timers instead.
4. **20–26 Oct:** code freeze 22 October; viva drills.

6 · Team conventions — agreed on the Friday 18 call

Four people are writing code that has to plug together on Monday 21. Each item below is a decision that, left unspoken, becomes a day of integration bugs. **On Friday 18 September at 8 pm, all four of us join a 30-minute call**, confirm or change each default, and Shaili commits the result as `docs/CONVENTIONS.md` on Saturday morning. Until then, treat these as proposals — and if you disagree with one, bring the reason to the call rather than quietly doing something else.

#	TOPIC	PROPOSED DEFAULT	OWNER
1	Protocol interface	One <code>ArqProtocol</code> interface with two operations: <i>send a file to a peer</i> and <i>receive a file on a port</i> . Both take a shared <code>ProtocolConfig</code> (window, RTO mode, sequence-space bits, payload size) and return <code>RunStats</code> . The session layer — metadata handshake, FIN teardown, hash check — sits outside the protocols and wraps whichever one is chosen, so Go-Back-N and Selective Repeat never reimplement it.	Shaili
2	Sequence numbers	The handshake is a DATA packet with <code>seq = 0</code> and flag bit <code>0x01</code> (META) set. File data starts at <code>seq = 1</code> . The sequence space is k bits, set with <code>--seqbits</code> ; the default is 32 and only wraparound tests make it small. Every comparison between sequence numbers uses modular arithmetic.	Shaili
3	What an ACK number means	The <code>ack</code> field always holds a sequence number that <i>was received</i> . Stop-and-Wait and Selective Repeat acknowledge exactly that packet. Go-Back-N's <code>ack n</code> is cumulative: everything up to and including n arrived in order — the textbook convention. Each protocol restates its rule in its class comment.	Shaili, Sinjan, Sohini
4	Payload size	1400 bytes per DATA packet; the last one is shorter. An 8 MiB file is therefore 5,992 data packets plus the handshake.	Shaili
5	Command-line flags	Sender: <code>--file</code> <code>--to host:port</code> <code>--protocol stopwait gbn sr</code> <code>--window N</code> <code>--rto adaptive fixed:X</code> <code>--seqbits k</code> <code>--verbose</code> . Receiver: <code>--port</code> <code>--out</code> <code>--protocol</code> <code>--window N</code> <code>--seqbits k</code> <code>--verbose</code> . An unknown flag is an error, never silently ignored.	Shaili
6	Ports	For manual runs the emulator listens on 9000 and the receiver on 9001. The experiment harness picks fresh ports for every run.	Sohan
7	Fixed RTO	<code>--rto fixed:1.5</code> means $\text{RTO} = 1.5 \times \text{base RTT}$, where base RTT = emulator delay up + delay down . With <code>--both delay=20</code> that is 40 ms, so <code>fixed:1.5</code> is 60 ms.	Sinjan
8	Result line	At the end of a transfer the sender prints exactly one line beginning <code>RESULT</code> followed by a single JSON object of <code>RunStats</code> fields. Nothing else may print a line starting with <code>RESULT</code> . <code>run_matrix.py</code> reads only that line.	Sohan
9	What goodput means	Goodput = $\text{file bytes} \times 8 \div \text{elapsed seconds}$, where <i>elapsed</i> runs from sending <code>seq 1</code> to receiving the ACK that covers the last data packet — handshake and FIN excluded. Wire throughput = every byte the sender put on the wire, headers and retransmissions included, $\times 8 \div$ the same elapsed time.	Sohan
10	RunStats fields	<code>protocol</code> , <code>window</code> , <code>seqbits</code> , <code>rto_mode</code> , <code>file_bytes</code> , <code>elapsed_ms</code> , <code>goodput_bps</code> , <code>wire_bytes</code> , <code>throughput_bps</code> , <code>data_sent</code> , <code>retransmissions</code> , <code>timeouts</code> , <code>dup_acks</code> , <code>acks_received</code> , <code>corrupt_dropped</code> , <code>sha256_match</code> . The harness adds the run settings itself — <code>seed</code> , <code>loss</code> , <code>dup</code> , <code>corrupt</code> , <code>reorder</code> , <code>delay</code> , <code>jitter</code> .	Sohan
11	Trace lines	<code>us</code> , <code>dir</code> , <code>type</code> , <code>seq</code> , <code>ack</code> , <code>action</code> , <code>delayMs</code> . The <code>type</code> and <code>ack</code> fields are new in the emulator rewrite. Today every ACK is logged with <code>seq 0</code> , so ACKs cannot be told apart — and Experiment 4 needs to tell them apart.	Sohan
12	Logging	No printing per packet unless <code>--verbose</code> is given. Writing a line for every packet costs more than the protocol itself and makes goodput numbers meaningless.	Everyone
13	Tests and main	Every new test class is added to both <code>build.sh</code> and the <code>TESTS</code> line of the <code>Makefile</code> , or it never runs. Run <code>./build.sh</code> before every push: <code>main</code> must always build, because a compile error there stops all four of us.	Everyone

#	TOPIC	PROPOSED DEFAULT	OWNER
14	Sequence arithmetic	One small <code>SeqSpace</code> helper for every modular comparison — distance from the window base, "inside the window", "inside the previous window" — built on <code>Math.floorMod</code> and shared by Go-Back-N and Selective Repeat. The wraparound rule is then written once and tested once.	Sohini

7 · How to test your work

TEST CLASS	WHAT IT MUST PROVE
<code>SeqSpaceTest</code>	Every line of the worked example in 3.5; <code>floorMod</code> gives 2 where <code>%</code> gives -2; the window and previous-window checks at both edges; a 32-bit space where the window straddles 4,294,967,295 and 0.
<code>ReceiveBufferTest</code>	In-order arrivals release immediately; 1, 3, 4, 2 releases 1 / nothing / nothing / 2, 3, 4; a duplicate of a stored packet is reported as a duplicate and not stored twice; a delivered packet arriving again is reported as previous-window; outside packets are ignored; a full window then filled from the base releases everything; wraparound across 6, 7, 0, 1 with a 3-bit space.
<code>SelectiveRepeatTest</code>	Correct transfer over all five integrity profiles; zero retransmissions on a clean channel ; completes with 20% of ACKs lost; with 10% data loss, retransmissions stay close to the number of data packets the emulator dropped; a <code>--seqbits 3</code> , window 4 transfer wraps and matches; a window of 5 with 3 bits is refused.

Demonstrate the illegal window at buffer level. The playbook asks you to show that an illegal window corrupts data. Doing that through the network means making the emulator lose *exactly* every ACK of one window, which is not reproducible. Instead, script the 3.4 scenario directly against `ReceiveBuffer` with a 2-bit space: window 3, deliver 0, 1, 2, then offer the old 0 and show it is accepted as new. Then the same with window 2, and show it is classified as previous-window. It is deterministic, it proves the same point, and the protocol itself refuses illegal windows — so there is no unsafe mode hiding in the real code.

Reuse the fixture. `NetEmSmokeTest` shows how to run the emulator inside a test on free ports.

Convention 13: add each new test class to `build.sh` and to the `Makefile`.

8 · When something breaks

SYMPTOM	LIKELY CAUSE	HOW TO CHECK
Transfer never finishes; sender's base stuck	Receiver ignoring previous-window packets instead of re-ACKing them	Run with 20% ACK loss; row 2 of the table in 3.2
Garbled or repeated blocks after a wrap	<code>%</code> instead of <code>Math.floorMod</code> , or window too large for <code>--seqbits</code>	<code>SeqSpaceTest</code> and the window validation
Retransmissions on a perfect channel	A cancelled timer fired and resent an acknowledged packet	The "still unacknowledged" check in 3.7
Retransmissions far above the losses	Timeout too short, or a timeout resending more than one packet	Compare with the trace's drop count
File bytes out of order	Delivering packets as they arrive rather than from the buffer head	<code>ReceiveBufferTest</code> 's 1, 3, 4, 2 case
Memory keeps growing	Storing outside-window packets, or never clearing delivered slots	Print occupied slots per second
<code>send_base</code> never slides although ACKs arrive	Sliding only by one, instead of past every consecutive acknowledged slot	Row 2 of the table in 3.1
A test passes on some seeds only	An assertion about timing instead of outcome	Assert on hashes and counts, never on exact timings
<code>bash\r: No such file or directory</code> in WSL	<code>build.sh</code> with Windows line endings	Re-clone inside WSL; <code>.gitattributes</code> now forces Unix line endings

9 · Experiment 2 — goodput against window size

This is the heart of our claim: the gap between Selective Repeat and Go-Back-N should widen as the window grows.

Question	How does goodput change with window size for Go-Back-N and Selective Repeat, and does the gap widen?
Variable	Window 1, 2, 4, 8, 16, 32, 64, 128
Held fixed	Loss 1% and 5% as two series; 1400-byte payload; 20 ms delay each way; 32-bit sequence space; file size, jitter and RTO mode as locked on Tue 22
Seeds	At least 3 per cell by Sun 27; 10 in the buffer week
Measured	<code>goodput_bps</code> , <code>retransmissions</code> , <code>throughput_bps</code>

Load check. The largest window sends at most 128 packets per 40 ms round trip — about 3,200 packets a second — an order of magnitude under the harness budget, so the curves reflect the protocols, not the machine. These runs take seconds to a minute each, unlike Stop-and-Wait.

What we expect

- At window 1, both protocols behave like Stop-and-Wait.
- Selective Repeat's goodput keeps rising with the window.
- Go-Back-N rises, then flattens or falls, because each loss can cost up to a whole window of resends — more noticeably at 5% than at 1%.
- The gap widens with the window, and faster at the higher loss rate.

Stay within what we can measure. Our emulator adds delay and loss but has no bandwidth limit, so there is no "bandwidth-delay product" knee in these curves. The proposal's claim is about loss rate and window size, which this experiment tests directly. Do not claim bandwidth effects in the report unless a rate limit is added to the emulator.

Figure 2

- Goodput against window, with the window axis doubling (logarithmic, base 2). Two panels — 1% and 5% loss — one line per protocol, 95% confidence-interval error bars.
- The caption states the settings and n .

10 · Your report section — Selective Repeat

1. **The window bound** — from `docs/sr-window-bound.md`: the rule, the failure, the legal case, and the buffer-level demonstration.
2. **Sender and receiver** — the tables in 3.1 and 3.2, and the lost-ACK deadlock that makes re-ACKing necessary.
3. **The receive buffer** — the design choice in 3.6 and why wraparound lives in `SeqSpace`.
4. **Timers** — per-packet timers and stale-timer safety; the fallback, if it was used.
5. **Experiment 2** — Figure 2 and what it shows about the claim.

About two to three pages.

11 · How we work together

Nightly check-in — by 10 pm, in the group chat

Three lines, every night of the sprint. For example:

```
Done:      receive buffer releases 1,3,4,2 in order; 5 tests passing
Tomorrow:  duplicate handling, then open the PR
Blocked:   nothing
```

In a two-week sprint, a silent day costs whoever is waiting on you half a day. If you are blocked, say so that evening — never save it for the next gate.

Branches

Never commit directly to `main`. Use one branch per task, named after your role — `m3/short-task-name`. Always start from the latest `main`:

```
git checkout main
git pull
git checkout -b m3/short-task-name
```

Bring `main` into your branch at least once a day, so you meet other people's changes while the conflicts are still small:

```
git fetch origin
git merge origin/main
```

Pull requests

Push your branch, open a pull request on GitHub, and use this description:

```
What:      one sentence
Why:      which playbook task this is
How tested: the commands you ran and what they printed
Ask me:    two things you want the reviewer to question
```

WHEN	YOUR REVIEWER — AND WHOSE PRS YOU REVIEW
Sprint A · 14–20 September	Sohan Mandal
Sprint B · 21–27 September	Shaili Seth

Review within 12 hours. An unreviewed PR blocks someone. If your partner is stuck, anyone else may review instead, and you review your partner's work on the same terms.

The explanation check. Before approving, the reviewer asks two or three *why* questions — *why this data structure, what happens if this ACK is lost, why this bound and not another*. If the author cannot answer, the PR does not merge. A five-minute voice call is enough. It rehearses the viva every day, and it catches copy-pasted code while there is still time to rewrite it.

Commits

- At least **one commit every working day** of the sprint, under your own name and email.
- Small commits whose messages say what changed — *"Release buffered packets once the gap at rcv_base fills"*, not *"update"*.
- Never commit on someone else's behalf. The brief counts that as non-contribution by the absent member.
- No Co-Authored-By lines, no bot pull requests, and Dependabot stays off.** The contributor list must show exactly our four names.

Other people's files

If you need a change in a file someone else owns, message them first. If it is urgent and they are unreachable, make the smallest change you can, say so in your PR, and tag them. A merge conflict in a file is resolved by that file's owner.

12 · Using AI without losing marks

PERMITTED BY THE BRIEF

Explaining concepts · debugging · reviewing your code · generating tests · boilerplate and plotting scripts · improving your writing

NOT PERMITTED

Generating the core protocol implementation and submitting it as your own work

Our rule: use AI to understand your module, then type the code yourself. Two reasons, both about marks. The viva is individual and worth 5 of the 10 marks, and sir will ask you to change your code live — code you did not write is code you cannot change under pressure, and it scores zero however well it runs. And a chatbot asked for a textbook protocol gives every group roughly the same code. Another group in our year is doing P3, and the brief treats identical-looking submissions as collusion.

Uses that help

- *"Explain Karn's algorithm with a timeline where the ACK arrives just after a retransmission."* Then close it and draw the timeline yourself.
- *"Here is my window check and a failing test. Why does it fail when the sequence numbers wrap?"* — debugging code you wrote.
- *"Write tests for a receive buffer that must release packets in order."* Tests are explicitly permitted.
- *"Quiz me on my module the way a viva examiner would."* The single most useful thing it can do for you.

Uses that cost marks

- *"Write GoBackN.java."* Even if you read it afterwards, you will not be able to change it live.
- Putting AI output into a PR you cannot then explain in the review check.

Log it the same day

Whenever AI materially helped, add an entry under your role in `AI-USE.md`:

```
### 2026-09-17 – receive buffer
**What:** asked why Java's % operator goes negative for a wrapped sequence number
**Permitted under:** explaining concepts
**Mine:** the buffer design, all of the code, and the tests for the wrap case
```

Honest disclosure carries no penalty. Undisclosed use discovered at the viva does.

13 · Viva preparation

1. Why does Selective Repeat need $W \leq 2^{k-1}$ when Go-Back-N only needs $W \leq 2^k - 1$?

- Sender window plus receiver window must fit in the sequence space; Go-Back-N's receiver window is 1, Selective Repeat's is W .
- Draw the $k = 2$, window 3 failure from 3.4, then the window 2 case working.

2. Why must the receiver ACK packets from the previous window?

- Delivered packet, lost ACK: the sender cannot advance and keeps resending. Without a fresh ACK it deadlocks.

3. How does the receiver decide whether a packet is new or a retransmission?

- Its offset from `rcv_base`: inside the window it may be new or a duplicate of a stored one; in the previous window it is old.
- This only works because of the window bound.

4. Why a timer per packet? What would a single timer lose?

- Each loss is recovered independently and in parallel.
- A single timer on the oldest packet recovers several losses one timeout at a time — the fallback in 3.9.

5. Why `Math.floorMod` and not `%`?

- `(1 - 3) % 4` is `-2` in Java; `floorMod` gives 2. Wraparound comparisons break with `%`.

6. Why a ring buffer rather than a `TreeMap`?

- $O(1)$, bounded memory, indexed by a wraparound-safe offset. A `TreeMap` sorts raw numbers, which misorders packets across a wrap.

7. Four packets, packet 2 lost. Count the retransmissions for each protocol.

- Walk the timeline in 3.3. Selective Repeat: 1. Go-Back-N: 3.

8. A timer fires for a packet that was already acknowledged. What happens?

- Nothing is sent: the fire handler checks the packet is still unacknowledged. Explain lazy cancellation.

9. Live change: shrink the window from 32 to 8 during a transfer.

- The sender stops sending past the new limit; slots already in flight are acknowledged or resent as normal.
- The receiver's window must match — and the new size must still satisfy the bound.

10. What would SACK blocks change on the wire?

- `sackCount` ranges of 32-bit start and end pairs in the payload; the header layout is unchanged.

14 · Help, references and glossary

Who to ask

QUESTIONS ABOUT	ASK	GITHUB
Packet format, checksum, handshake, FIN, CLIs, Stop-and-Wait, the integrity suite	Shaili Seth	shailiseth24
Timers, RTT estimation, Karn's algorithm, Go-Back-N, fixed and adaptive RTO	Sinjan Mishra	Thesin-01
Selective Repeat, the receive buffer, window bounds, wraparound	Sohini Pandit	sohinix
The emulator, trace files, calibration, RunStats, CSVs, plots, repository access	Sohan Mandal	sohan1611

Stuck for more than an hour? Say so in the group chat. An hour stuck in a two-week sprint is expensive; two minutes of someone else's attention usually is not.

References

- **RFC 1071**, *Computing the Internet Checksum* — section 1 is the algorithm.
- **RFC 6298**, *Computing TCP's Retransmission Timer* — Jacobson/Karels estimation, Karn's algorithm, backoff.
- **Kurose & Ross**, *Computer Networking: A Top-Down Approach* — section 3.4 (reliable data transfer: rdt 3.0, Go-Back-N, Selective Repeat) and section 3.5 (TCP round-trip time estimation).
- **Java 21 API documentation** — [java.nio.ByteBuffer](#), [java.net.DatagramSocket](#), [java.util.PriorityQueue](#), [java.security.MessageDigest](#), [java.util.Random](#), [Math.floorMod](#).
- **In the repository** — [docs/PLAN.md](#) for the design decisions, [docs/member-playbook.html](#) for everyone's dates, [AI-USE.md](#), and the existing tests, which show the house test style.

Glossary

ARQ	Automatic Repeat reQuest: reliability by detecting loss and retransmitting. Stop-and-Wait, Go-Back-N and Selective Repeat are three ARQ schemes.
RTT	Round-trip time — data out, ACK back.
RTO	Retransmission timeout — how long the sender waits for an ACK before resending.
SRTT, RTTVAR	The smoothed RTT and its smoothed variation, from which RTO is computed (RFC 6298).
Cumulative ACK	"Everything up to n has arrived." Used by Go-Back-N.
Selective ACK	"This particular packet has arrived." Used by Selective Repeat.
Window	How many unacknowledged packets the sender may have in flight at once.
Sequence space	The range of sequence numbers: 2^k values for k bits, wrapping back to 0.
Goodput	Useful file bytes delivered per second — excluding headers and retransmissions.
Throughput	Every byte put on the wire per second — including headers and retransmissions.
Spurious retransmission	Resending a packet whose original actually arrived.
Network byte order	Big-endian: most significant byte first. <code>ByteBuffer</code> 's default.
One's complement sum	Addition where a carry out of the top bit is added back in at the bottom — the end-around carry. The basis of the Internet checksum.
Seed	The starting value for the emulator's random generator. Same seed, same decisions, on any machine.
95% confidence interval	A range around a measured mean reflecting run-to-run variation. Every data point we plot carries one.
JIT warm-up	The JVM runs code slowly at first and compiles hot paths as it goes, so the first packets of a run can be slower.

15 · Sprint checklist

- ☐ **Tue 15** — cloned, identity set, build green, first commit merged
- ☐ **Tue 15** — the window-bound failure drawn from memory
- ☐ **Wed 16** — `docs/sr-window-bound.md` committed; SeqSpace PR open
- ☐ **Thu 17** — reviewed Sohan's emulator PR
- ☐ **Fri 18** — SeqSpace and ReceiveBuffer merged, wraparound cases passing; joined the 8 pm call
- ☐ **Sat 19** — sender design on paper; read CONVENTIONS.md
- ☐ **Mon 21** — Selective Repeat transfers at 0% loss
- ☐ **Tue 22** — only lost packets resent at 10% loss; fallback decided by 10 pm
- ☐ **Wed 23** — 3-bit wraparound run matches; illegal window refused; buffer-level demonstration
- ☐ **Thu 24** — Selective Repeat merged and passing the integrity suite: Core complete
- ☐ **Sun 27** — Figure 2 from at least 3 seeds per cell
- ☐ **Every night** — three-line check-in by 10 pm; at least one commit every working day